AI WeatherWise - Smart weather tracking with AI-powered guidance.

Description

AI WeatherWise is a restful backend service built on Node.js and Express.js, with MongoDB and Mongoose ODM handling storage. It pulls live weather readings and passes them to Google Gemini AI, which turns raw numbers into short plain-language summaries and practical suggestions tailored to what the sky is doing right now.

Access is protected through JWT-based authentication and bcrypt-hashed passwords, and the API offers a full set of endpoints for saving favorite cities and requesting weather insights. A built-in fallback mode keeps the service responding even when external API keys are missing, while shared error-handling middleware and input sanitization keep the whole application predictable and safe.

Scenario-Based Case Study

Background

Priya is a weekend trekker and part-time photographer who travels to a new town almost every month. Before each trip she wants a quick, reliable view of the local weather and clear advice on what to pack and which activities suit the day.

Problem

- Weather for several destinations ends up scattered across different apps and bookmarks.
- Raw readings offer no guidance on what to wear or which outdoor plans make sense.
- Numbers such as humidity and wind speed are hard to interpret without a readable summary.
- Saved destinations are not stored securely or kept consistent across phone and laptop.

Solution

AI WeatherWise acts as one central service that fetches current conditions and coordinates the AI layer. It keeps secure records of users and their saved locations, then asks Google Gemini to produce summaries and recommendations that fit each location's live weather.

Usage

- **Registered User:** Adds, views, updates and removes favorite cities, checks current weather, and asks the AI for a summary or activity and clothing advice.
- **Administrator:** Keeps an eye on service health, oversees the pool of registered accounts, and confirms that the weather and Gemini integrations are working.
- **Public Consumer:** Calls the open weather endpoints from a website or app to display current conditions with no sign-up needed.

Outcome

With the heavy lifting handled by AI and one central database, users no longer have to interpret numbers themselves. Mongoose schemas keep stored data consistent, and Gemini delivers clear, situation-aware advice for any saved city.

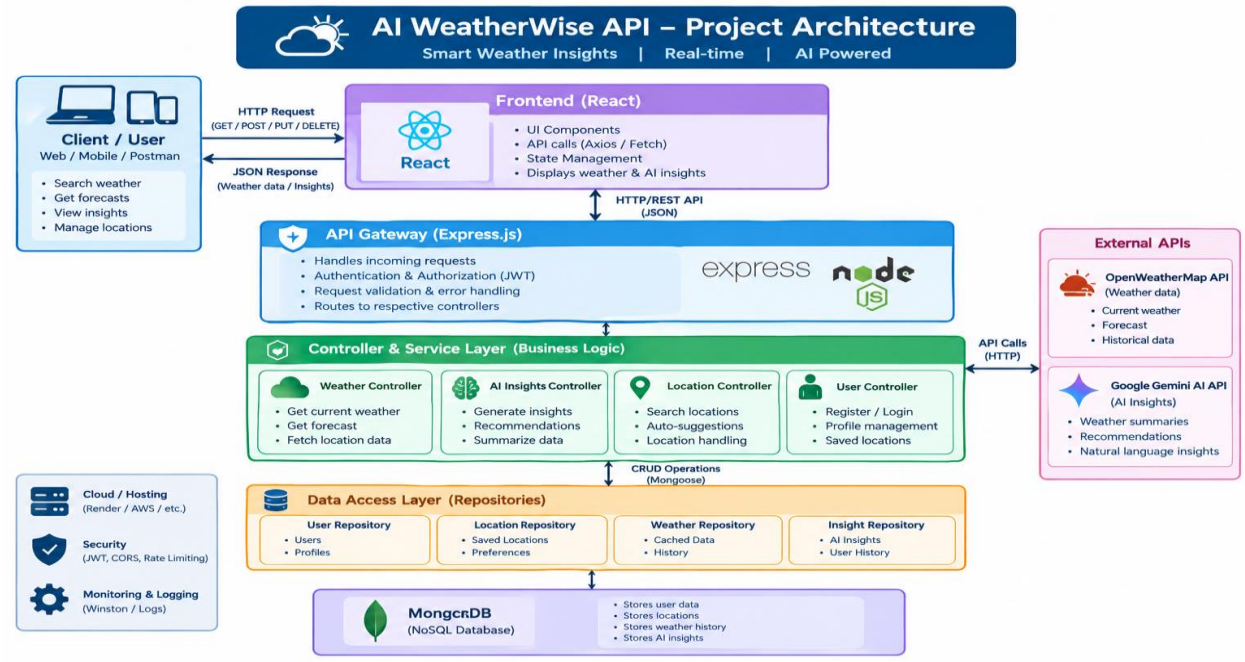
1. Software Requirements

- **Operating System:** Windows 10/11, macOS or a modern Linux distribution.
- **Node.js (v16 or newer):** JavaScript runtime that executes the server code and routing.
- **npm (v8 or newer):** Package manager used to install and maintain project libraries.
- **Gemini AI SDK:** The Google generative AI client library used to request summaries and recommendations.
- **MongoDB:** NoSQL document database that stores user accounts and saved locations.
- **Postman:** API client for sending test requests and checking responses from each endpoint.
- **Code Editor:** Visual Studio Code or any comparable editor.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th generation or later) or AMD Ryzen 5, or better.
- **RAM:** 8 GB minimum; 16 GB is recommended when running MongoDB, the server and testing tools together.
- **Storage:** At least 1 GB of free disk space.

- # Epic 1: PROJECT ARCHITECTURE

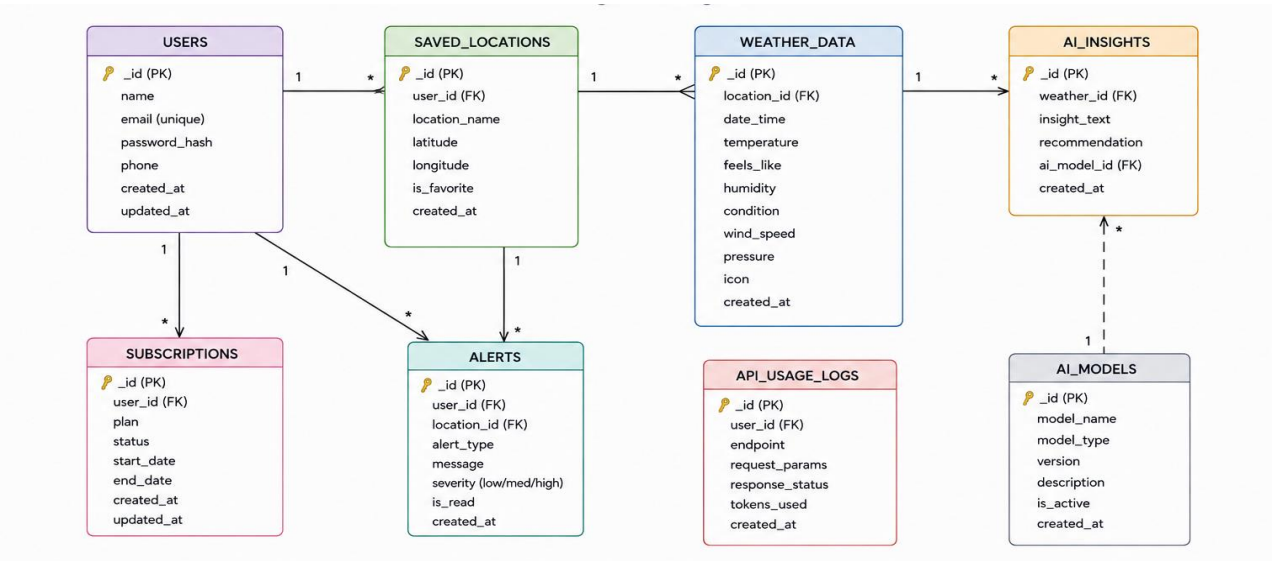


AI WeatherWise is organized using the MVC pattern so that business rules stay separate from data handling. Controllers retrieve conditions from OpenWeatherMap and request insights from Gemini, while models define and store the User and Location records.

Components Definition

- **Client Interface:** Tools such as Postman send requests to the API routes and receive JSON responses.
- **Express Server:** Listens on the configured port, routes GET, POST, PUT and DELETE requests, parses JSON bodies and applies CORS settings.
- **Authentication Middleware:** Reads the token from the request header, verifies its signature, and decides whether the caller may reach a protected route.
- **Service Modules:** Contain the core logic for managing saved locations, calling the weather provider and preparing data for the AI prompt.
- **Database Layer:** Mongoose converts application operations into MongoDB queries and enforces schema rules.
- **AI Service Integration:** Uses the Google generative AI library to create weather summaries and tailored suggestions from the current temperature and conditions.

Entity-Relationship (ER) Diagram Description



The data design links each User to the favorite Locations they save, using a simple reference between the two collections: `Users.id` → `Saved_Locations.user_id`

1. User Entity

- _id (Primary Key, ObjectId)
- name (String, required)
- email (String, required, must be unique)

- password (String, required, stored as a bcrypt hash)
- role (String, defaults to "reader")

2. Location Entity

- _id (Primary Key, ObjectId)
- user (Reference to the User collection)
- city (String, required)
- country (String, required)
- createdAt (Date, defaults to the current time)

Key Features

- **Protected Routes:** Token checks keep private endpoints closed to unauthenticated visitors.
- **Favorite Location Management:** Full create, read, update and delete operations for each user's saved cities.
- **Live Weather Retrieval:** Current temperature, humidity and conditions fetched on demand for any saved city.
- **AI Insights:** Gemini generates readable summaries and suggestions for clothing and outdoor activities.
- **Security and Resilience:** Input sanitization guards against XSS and NoSQL injection, and a fallback mode keeps responses flowing when API keys are unavailable.

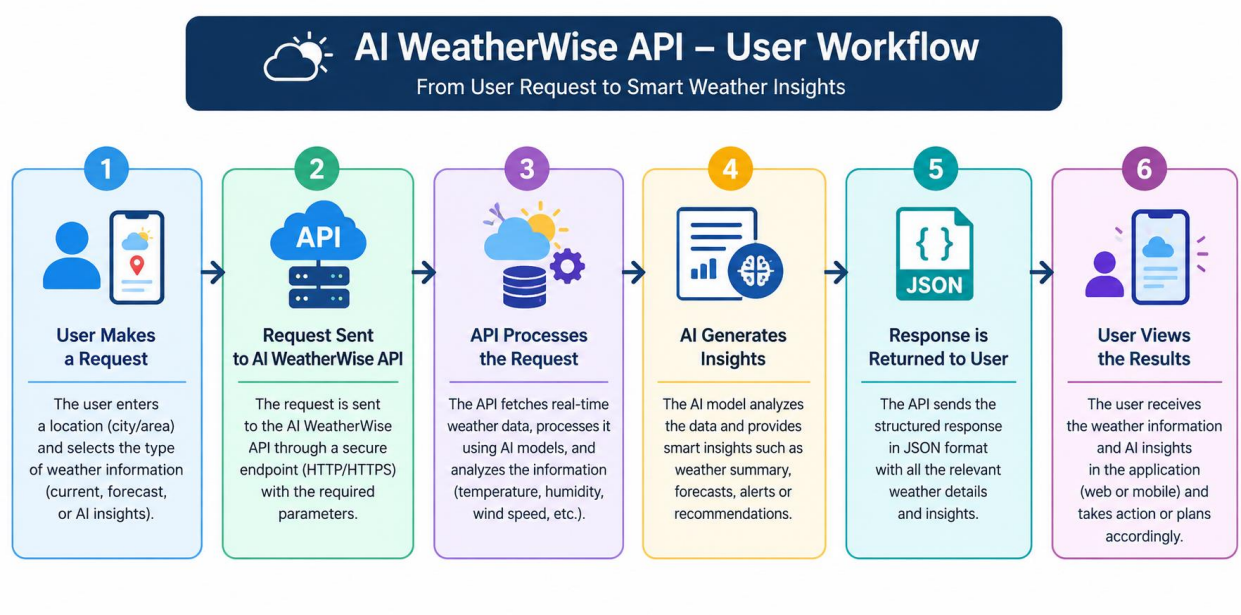
Roles and Responsibilities

- **Administrator:**
  - Manages user accounts, including creating, removing or suspending them.
  - Watches API health and system logs to track performance.
  - Maintains the integrity of the stored data.
- **Registered User:**
  - Builds and updates a personal list of favorite cities.
  - Requests up-to-date weather for those cities.
  - Uses the Gemini-powered endpoints for summaries and activity suggestions.
  - Controls their own account details, such as editing profile information.

Typical User Flow:

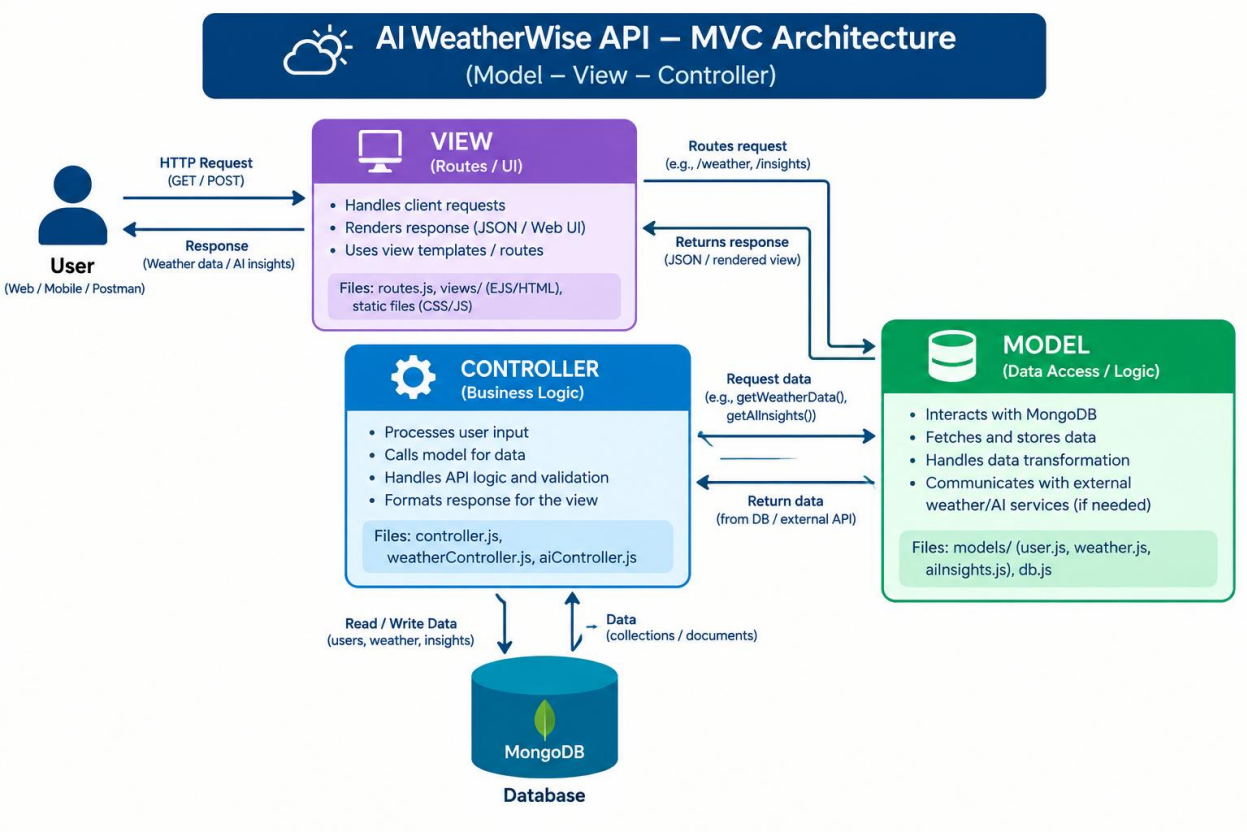
1. The user signs up or logs in.
2. The user stores a favorite city through the API.
3. The user requests the latest weather for that city.

4. The user asks for an AI summary or recommendations built from that weather.
5. The server verifies the JWT and sends back the response.



MVC Architecture Pattern

The codebase follows the Model-View-Controller (MVC) pattern, which keeps each responsibility in its own layer. The Model handles data and database operations using Mongoose and MongoDB. The View represents the client/API response layer, such as JSON responses. The Controller handles incoming requests, validation, and application logic, and communicates with the Model and external services such as OpenWeatherMap and Google Gemini.



Model Layer (Data Structure)

Mongoose schemas describe the shape of each collection, including required fields, defaults and indexes, and validate documents before they are saved to MongoDB.

Controller Layer (Request Handling)

Controllers receive requests from the routes, validate the input, call the models and external services, and return results as consistent JSON responses.

View Layer (Routing Interface)

Because this is a headless backend, no HTML templates are rendered. The "view" is the routing layer, which maps each URL and HTTP method to the matching controller function.

Epic 2: Project Setup and Configuration

Creating the Project Folder

- Create a new folder named **AI WeatherWise**
- Open the folder in **Visual Studio Code**.
- Set up a Node.js project and install the libraries the application needs.

Project Initialization

In the VS Code terminal, inside the project folder, run:

```
npm init -y
```

Install the runtime dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

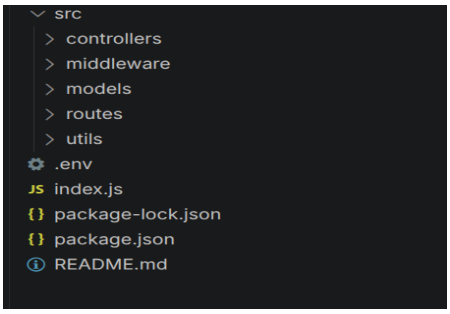
Add Nodemon as a development tool:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



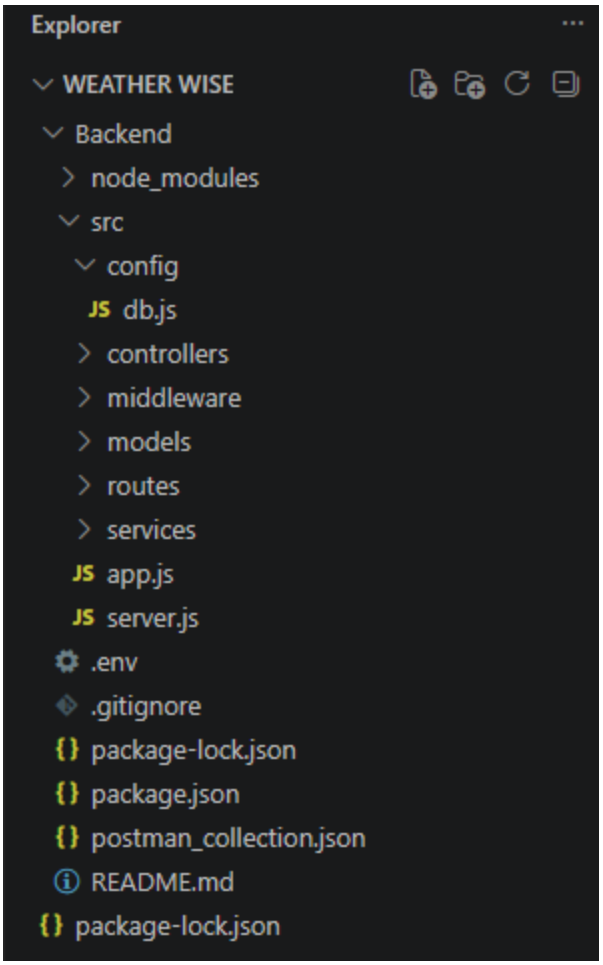
Environment Configuration

Add a **.env** file in the project root containing the following settings:

```
PORT=5001
MONGO_URI=mongodb+srv://inbanathan2006_db_user:vb88LC0EAP7EKw7B@cluster0.5kmbjn3.mongodb.net/WeatherAIAI?appName=Cluster0
JWT_SECRET=your_super_secret_jwt_key_12345!
OPENWEATHER_API_KEY=0ea7c96bd871a4a1c43b317b62feb0a4
GEMINI_API_KEY=AQ.Ab8RN6KHp5HpDmZHsCKVQuUvf__9UjWtx1iCYcgeQjjLQLUtDA
```

Epic 3: Backend Development

The WeatherWise backend is written with **Node.js**, **Express.js** and **MongoDB** in a modular layout. Routes, controllers, middleware, models and utilities live in separate folders, which makes the code easier to maintain and extend. The service exposes secure REST endpoints for authentication, location management, weather lookups and AI-generated insights.



Core System Script Implementations

1. Application Entry Point (index.js)

The **index.js** file is the starting point of the application. It loads environment variables, creates the Express app, applies middleware, connects to MongoDB, registers the API routes, and begins listening for requests once the database connection succeeds.

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const authRoutes = require('./routes/authRoutes');
const locationRoutes = require('./routes/locationRoutes');
const weatherRoutes = require('./routes/weatherRoutes');
const aiRoutes = require('./routes/aiRoutes');

const app = express();

// Middlewares
app.use(cors());
app.use(express.json());

// API Base Route
app.get('/', (req, res) => {
  res.json({
    message: 'Welcome to the AI WeatherWise API!',
    documentation: 'Use Postman collection or check README.md for endpoint information'
  });
});
```

Epic4: Database Configuration

1. Configure MongoDB:

- Install Mongoose.
  In your Node.js project directory, run:
  npm install mongoose
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Before any request is processed, the server should confirm that the database connection is active. The connection code is similar to the example below:

Mongoose Schema Definitions

Location Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

```

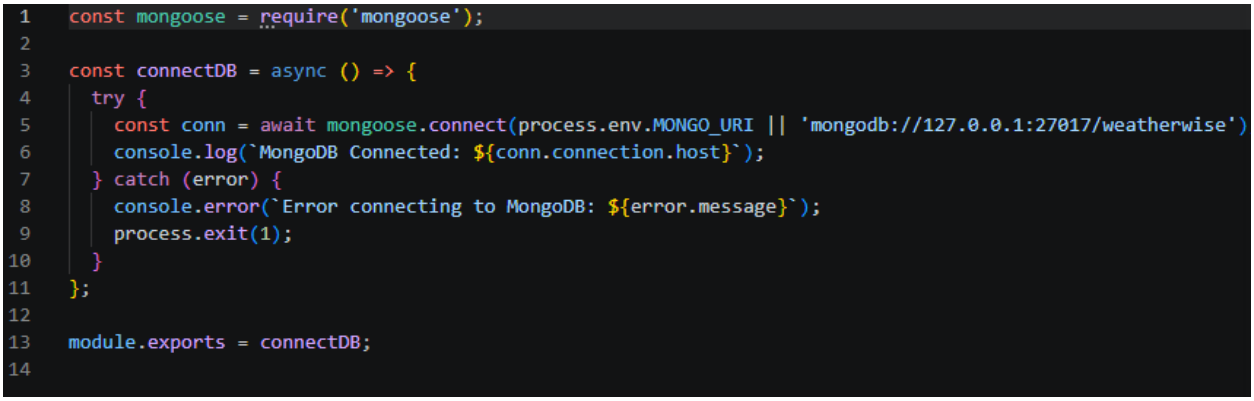
User schema:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
```

Db connection:

This module links the application to MongoDB through Mongoose, making sure the database is reachable before the server accepts client requests.



```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise')
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;

```

Epic 5: Project Executions:

The backend can be run on a local machine with Node.js. Make sure MongoDB is running and the environment variables are set, then use Postman or Thunder Client to try the endpoints.

Running the Backend Server

Install every dependency listed in package.json:

```
npm install
```

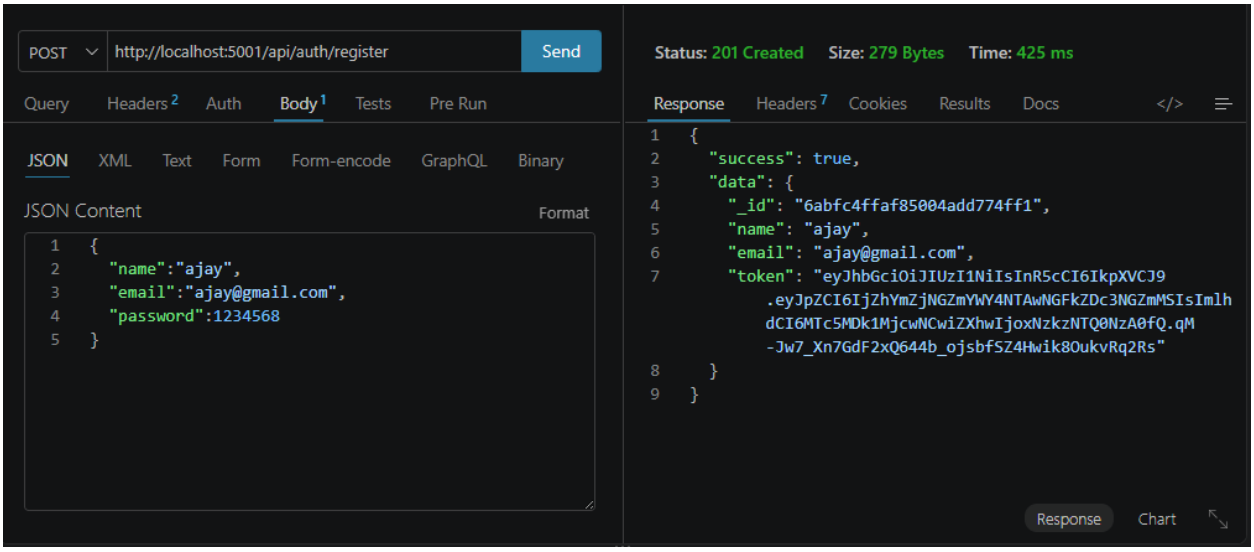
Launch the server:

```
npm start
```

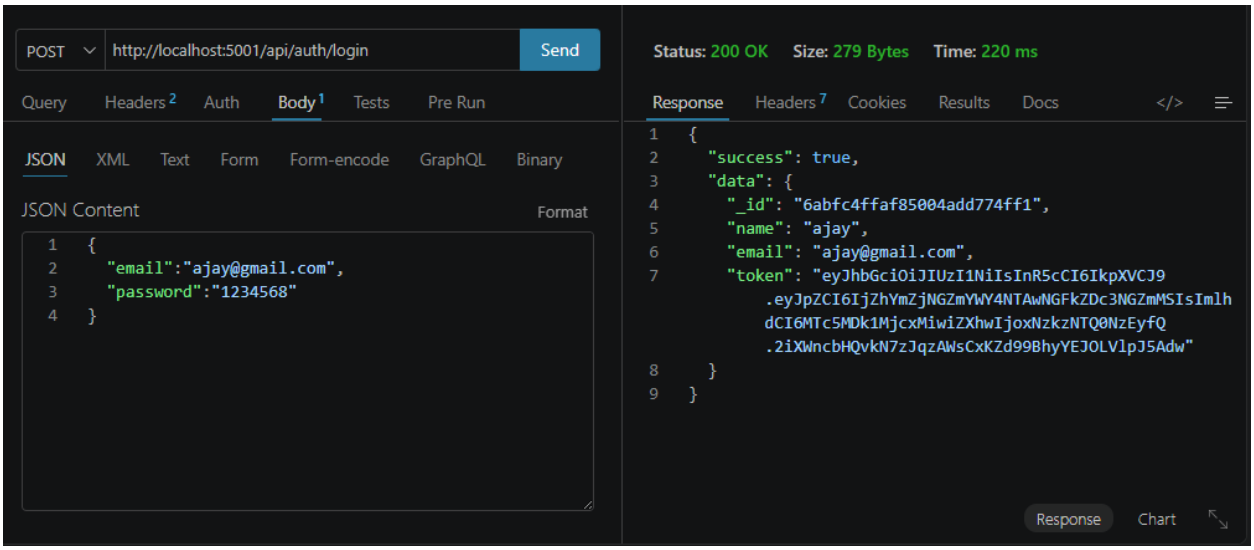
Once started, the server connects to MongoDB and waits for incoming API calls.

API testing:

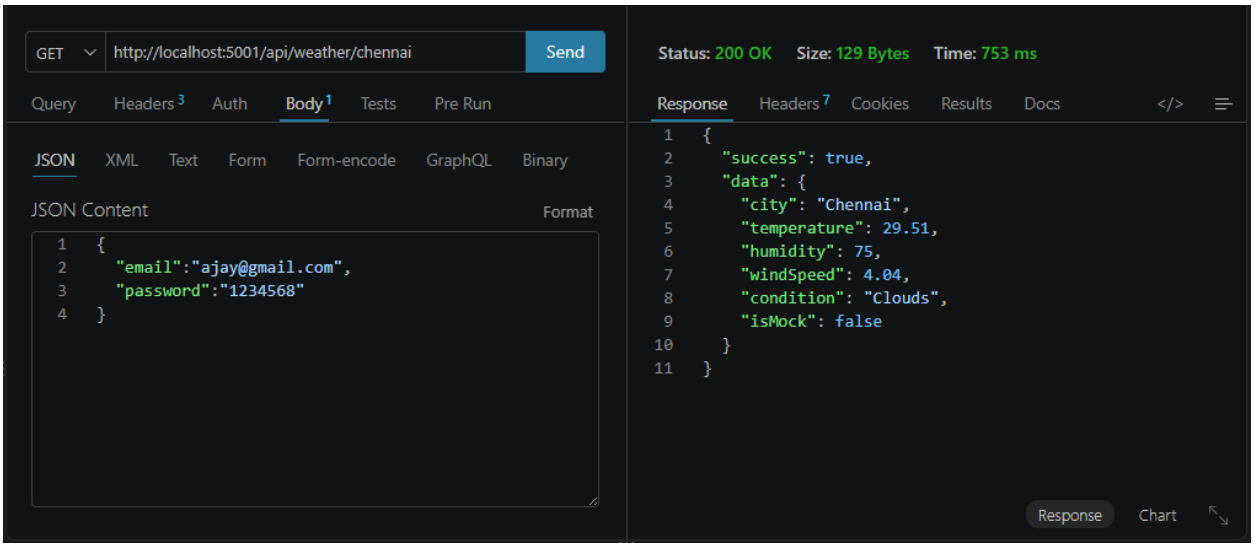
Register API:



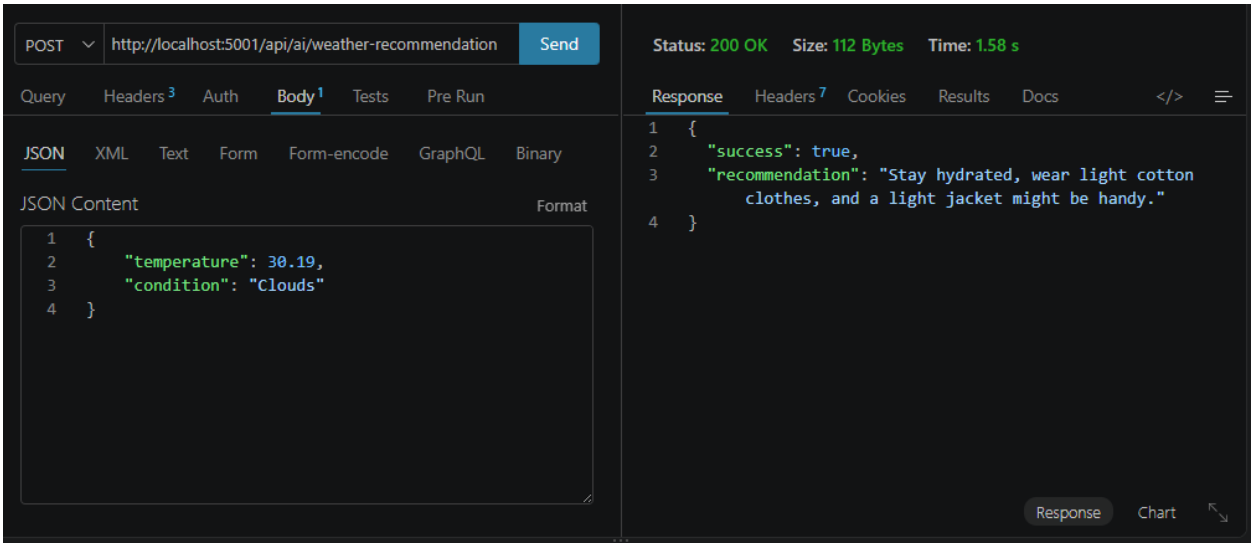
Login API:



Weather API:



Recommendation API:



Demo Video Link:

https://drive.google.com/file/d/14C6FAm2nB6x9sgkPPTchBMsyxAyGeNOb/view?usp=sharing

CodeDrive: https://drive.google.com/drive/folders/10Kd3ldGtjAgaSwgihMDVU2qL2TAl2K2T